

NARROW RECON METHODOLOGY

فهرست مطالب

پیش از شروع — چکلیست آمادگی عملیاتی (Pre-Engagement)

مرحله ۱ — شناخت دستی اپلیکیشن (Manual Recon)

مرحله ۲ — تحلیل پاسخ‌های سرور (HTTP Analysis)

مرحله ۳ — بررسی گواهی و رمزنگاری ارتباط (SSL/TLS Analysis)

مرحله ۴ — شناسایی فناوری (Technology Fingerprinting)

مرحله ۵ — خزش سایت (Web Crawling)

مرحله ۶ — شناسایی در کد سمت کاربر (JavaScript Recon)

مرحله ۷ — کشف محتوا (Content Discovery)

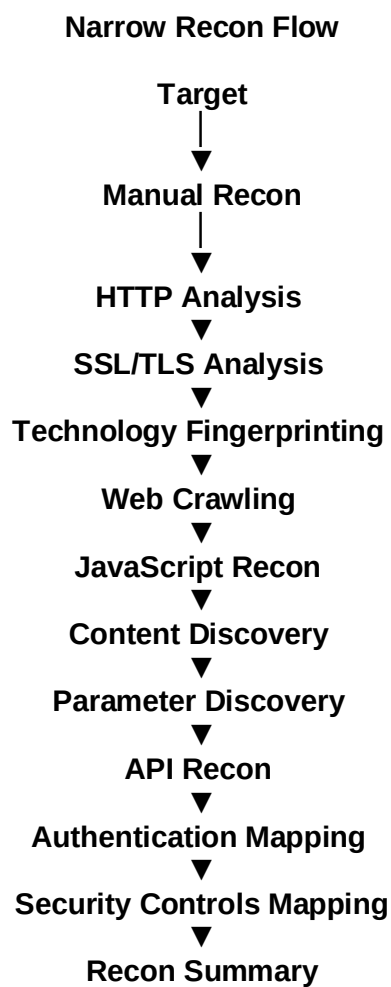
مرحله ۸ — کشف پارامترها (Parameter Discovery)

مرحله ۹ — شناسایی رابط برنامه‌نویسی (API Recon)

مرحله ۱۰ — نقشه‌برداری احراز هویت (Authentication Mapping)

مرحله ۱۱ — شناسایی کنترل‌های امنیتی (Security Controls Mapping)

مرحله ۱۲ — جمع‌بندی نهایی (Recon Summary)



پیش از اجرای هر مرحله از این سند موارد زیر باید تأیید شده باشند

(Pre-Engagement Checklist) چک‌لیست آمادگی عملیاتی

- ☐ مجوز کتبی برای انجام تست دریافت شده است
- ☐ محدوده دقیق تست (Scope) مشخص و مستند شده است
- ☐ بخش‌های خارج از محدوده (Out of Scope) مشخص شده‌اند
- ☐ دامنه‌ها، زیردامنه‌ها یا IPهای مجاز برای تست مشخص شده‌اند
- ☐ نوع تست (Black Box / Gray Box / White Box) مشخص شده است
- ☐ حساب‌های کاربری و سطح دسترسی موردنیاز (در صورت وجود) دریافت شده است
- ☐ بازه زمانی مجاز برای اجرای تست تعیین شده است
- ☐ قالب ثبت یافته‌ها و محل ذخیره‌سازی مستندات مشخص شده است

مرحله 1 — شناخت دستی اپلیکیشن

Manual Recon

هدف	شناخت کامل رفتار، ساختار، قابلیت‌ها و جریان‌های اصلی اپلیکیشن از طریق بررسی دستی، پیش از اجرای هرگونه ابزار خودکار.
چرا این مرحله؟	اجرای زود هنگام ابزارها بدون شناخت اپلیکیشن باعث هدررفت زمان، تولید خروجی‌های غیرضروری و از دست رفتن جزئیات رفتاری می‌شود که تنها با تعامل مستقیم با برنامه قابل مشاهده هستند.

چک‌لیست بررسی دستی

ساختار کلی اپلیکیشن

- ☐ صفحه اصلی و عملکرد کلی سایت
- ☐ صفحات اصلی و منوهای ناوبری
- ☐ ساختار و الگوی URL ها
- ☐ صفحات خطا 403، 404، 500
- ☐ رفتار Redirect ها

احراز هویت

- ☐ وجود یا عدم وجود Login
- ☐ وجود یا عدم وجود Register
- ☐ فرآیند Logout
- ☐ فرآیند Forgot Password
- ☐ فرآیند Change Password
- ☐ وجود یا عدم وجود MFA

حساب کاربری

- ☐ پروفایل کاربری

□ صفحه تنظیمات

□ انواع نقش‌های کاربری Admin ، User ، Guest و...

□ تفاوت قابلیت‌های هر نقش (در صورت امکان)

قابلیت‌های اپلیکیشن

□ فرم‌های موجود

□ جستجو (Search)

□ بارگذاری فایل (Upload)

□ دانلود فایل (Download)

□ Import

□ Export

□ اعلان‌ها (Notifications)

□ صفحات تماس یا ارسال بازخورد

نقاط ورود اطلاعات (Input Points)

□ تمام فیلدهایی که کاربر می‌تواند داده وارد کند شناسایی شوند.

مانند:

- Login
- Search
- Contact Form
- Comment
- Profile
- Upload
- Address
- Feedback

فایل‌ها و صفحات عمومی

□ وجود robots.txt

□ وجود sitemap.xml

□ وجود security.txt

□ وجود favicon.ico

فقط وجود یا عدم وجود آن‌ها ثبت شود؛ تحلیل تخصصی این فایل‌ها در مراحل بعد انجام می‌شود.

سایر مشاهدات

□ زبان‌های قابل انتخاب

□ رفتار Session بعد از Logout

□ پیام‌های موفقیت و خطا

□ فایل‌های قابل دانلود

□ لینک‌های خارجی و سرویس‌های شخص ثالث (در صورت مشاهده)

بهترین شیوه اجرا

ابتدا اپلیکیشن بدون ورود به حساب کاربری بررسی شود. سپس در صورت وجود حساب کاربری، با یک کاربر عادی وارد سیستم شوید و تفاوت رفتار، صفحات و قابلیت‌ها قبل و بعد از احراز هویت را مستند کنید. در صورت دسترسی به نقش‌های مختلف، تفاوت قابلیت‌ها و مسیرهای قابل دسترسی هر نقش نیز بررسی شود.

خروجی مورد انتظار

- نقشه اولیه صفحات و ساختار اپلیکیشن
- فهرست قابلیت‌های موجود
- فهرست نقاط ورود اطلاعات (Input Points)
- نقش‌های کاربری شناسایی‌شده
- صفحات و قابلیت‌های نیازمند بررسی بیشتر
- سطح حمله اولیه (Initial Attack Surface)

مرحله 2 — تحلیل پاسخ‌های سرور

HTTP Analysis

هدف	بررسی Header های HTTP ، Cookie ها، مکانیزم‌های امنیتی مرورگر و رفتار پاسخ‌دهی سرور به درخواست‌های مختلف، به منظور شناسایی اطلاعات اولیه درباره پیکربندی و سطح امنیت ارتباط.
چرا این ابزار؟	سبک‌ترین و شفاف‌ترین راه برای مشاهده خام پاسخ سرور است، بدون واسطه مرورگر یا رابط curl. گرافیکی اضافه به همین دلیل تنها ابزار این مرحله است.
ابزار پیشنهادی	curl

دستور اجرا 1

```
curl -s -D - -o /dev/null https://target.com
```

توضیح دستور

- -s : اجرای درخواست در حالت Silent و حذف نمایش پیام‌های اضافی مانند Progress Meter.
- -D - : نمایش Header های پاسخ HTTP در خروجی ترمینال - به معنی ارسال Header ها به خروجی استاندارد است
- -o /dev/null : نادیده گرفتن و عدم ذخیره Body پاسخ؛ در نتیجه فقط Header های HTTP مشاهده می‌شوند

دستور اجرا 2

```
curl -H "Origin: https://evil.com" -I https://target.com
```

توضیح دستور

- ارسال هدر Origin سفارشی برای بررسی رفتار CORS
- -I : دریافت فقط هدرهای پاسخ

بهترین شیوه اجرا

ابتدا روی صفحه اصلی و سپس روی صفحات حساس مانند ورود و پنل مدیریت اجرا شود تا اختلاف رفتار هدرها مشخص شود

□ Header های پاسخ HTTP بررسی شدند.

□ Server Header بررسی شد.

□ Cookie های ایجادشده بررسی شدند.

□ ویژگی های Secure، HttpOnly و SameSite بررسی شدند.

□ Security Header ها بررسی شدند.

□ رفتار CORS بررسی شد.

□ تفاوت پاسخ صفحات عمومی و حساس بررسی شد.

□ متدهای HTTP قابل مشاهده ثبت شدند.

خروجی مورد انتظار:

- فهرست HTTP Header های پاسخ
- اطلاعات Cookie و ویژگی های امنیتی آن
- وضعیت Security Header ها
- رفتار CORS
- متدهای HTTP قابل مشاهده
- تفاوت رفتار صفحات عمومی و حساس

مرحله 3 — بررسی گواهی و رمزنگاری ارتباط

SSL/TLS Analysis

هدف

ارزیابی پیکربندی SSL/TLS شامل نسخه های فعال پروتکل، مجموعه های رمزنگاری (Cipher Suites)، اطلاعات گواهی دیجیتال و تنظیمات امنیتی ارتباط، به منظور شناسایی وضعیت کلی امنیت لایه انتقال.

چرا این ابزار؟

Nmap با استفاده از اسکریپت های NSE امکان بررسی نسخه های TLS، مجموعه های رمزنگاری و اطلاعات گواهی دیجیتال را در یک اجرا فراهم می کند. این ابزار سبک، قابل اعتماد و مناسب برای فاز Recon بوده و به راحتی روی سیستم عامل های مختلف از جمله ویندوز قابل استفاده است.

ابزار پیشنهادی

Nmap (NSE Scripts)

دستور اجرا ۱ — بررسی نسخه های TLS و Cipher Suite ها

```
nmap --script ssl-enum-ciphers -p 443 target.com
```

توضیح دستور

- `--script ssl-enum-ciphers`: نمایش نسخه های TLS و Cipher Suite های قابل استفاده
- `-p 443`: بررسی سرویس HTTPS روی پورت ۴۴۳

دستور اجرا ۲ — بررسی اطلاعات گواهی

```
nmap --script ssl-cert -p 443 target.com
```

توضیح دستور

- `--script ssl-cert`: استخراج اطلاعات گواهی دیجیتال
- `-p 443`: بررسی سرویس HTTPS روی پورت ۴۴۳

بهترین شیوه اجرا

بررسی روی دامنه اصلی و در صورت وجود سرویس‌های حساس مانند **API**، **پنل مدیریت** و **سابدامین‌های مهم** نیز تکرار شود. در صورت مشاهده تفاوت در نسخه‌های TLS یا گواهی، هر سرویس به‌صورت جداگانه مستندسازی شود.

- نسخه‌های TLS فعال ثبت شدند.
- نسخه‌های قدیمی TLS بررسی شدند.
- Cipher Suite های فعال بررسی شدند.
- اطلاعات Certificate استخراج شد.
- تاریخ انقضا Certificate بررسی شد.
- صادرکننده (Issuer) و دامنه‌های موجود در Certificate بررسی شدند.
- تفاوت تنظیمات SSL/TLS بین سرویس‌های مختلف بررسی شد.

خروجی مورد انتظار

- نسخه‌های TLS فعال
- Cipher Suite های قابل استفاده
- اطلاعات Certificate (Subject، Issuer و تاریخ اعتبار)
- دامنه‌های موجود در Certificate (SAN)
- وضعیت کلی پیکربندی SSL/TLS
- سرویس‌های دارای تنظیمات متفاوت یا نیازمند بررسی بیشتر

مرحله 4 — شناسایی فناوری

Technology Fingerprinting

هدف

شناسایی فناوری‌های استفاده‌شده در اپلیکیشن، شامل وب‌سرور، سیستم مدیریت محتوا (CMS)، فریمورک‌ها، زبان برنامه‌نویسی، کتابخانه‌ها، CDN و سایر فناوری‌هایی که در مراحل بعدی تست نفوذ برای انتخاب تکنیک‌های مناسب مورد استفاده قرار می‌گیرند.

چرا این ابزار؟

WhatWeb با استفاده از پایگاه داده گسترده‌ای از امضاها (Signatures) می‌تواند تنها با یک اجرا بسیاری از فناوری‌های مورد استفاده در وبسایت را شناسایی کند و اطلاعات مناسبی برای مرحله Recon فراهم کند.

ابزار پیشنهادی

WhatWeb

دستور اجرا

```
ruby whatweb -v https://target.com
```

توضیح دستور

- V: نمایش خروجی کامل شامل تمام فناوری‌های شناسایی‌شده و جزئیات بیشتر

بهترین شیوه اجرا

- ابتدا روی صفحه اصلی اجرا شود.
- سپس روی صفحات مهم مانند Login، Dashboard یا پنل مدیریت نیز اجرا شود.

□ وب‌سرور (Apache / Nginx / IIS ...)

□ سیستم مدیریت محتوا (CMS)

□ فریمورک Backend

□ از زبان برنامه‌نویسی

□ فریم‌ورک یا کتابخانه Frontend

□ نسخه فناوری‌ها (در صورت نمایش)

□ CDN

□ WAF در صورت قابل تشخیص بودن

□ اطلاعات افشاشده در Header ها یا Banner ها

خروجی مورد انتظار

- Web Server
- CMS
- Programming Language
- Backend Framework
- Frontend Framework / Library
- Technology Versions در صورت وجود
- CDN
- WAF در صورت شناسایی
- سایر فناوری‌های شناسایی‌شده

۱. WAF

گاهی WhatWeb می‌تواند وجود WAF را تشخیص دهد. این اطلاعات برای مراحل بعدی بسیار مهم است، چون روی نوع تست و نرخ درخواست‌ها تأثیر می‌گذارد.

۲. نسخه فناوری‌ها

اگر 2 آسیب‌پذیری‌های شناخته‌شده تطبیق داد. حتی اگر همیشه نسخه را پیدا نکنی، بهتر است در چک‌لیست و خروجی مورد انتظار جایی برای ثبت آن داشته باشی.

مرحله 5 — خزش سایت

Web Crawling

هدف	جمع‌آوری URL های قابل دسترس، Endpoint های موجود، فایل‌های JavaScript، منابع سمت کاربر و ساختار داخلی برنامه از طریق دنبال کردن لینک‌ها و منابع قابل مشاهده.
چرا این ابزار؟	Katana یک ابزار مدرن برای Web Crawling است که قابلیت خزش بازگشتی، استخراج URL ها از منابع مختلف، پردازش JavaScript را فراهم می‌کند.
ابزار پیشنهادی	Katana

دستور اجرا

```
katana -u https://target.com -d 3 -jc -o urls.txt
```

توضیح دستور

- -u: تعیین URL هدف
- -d 3: تعیین عمق خزش تا سه سطح

- -jc: استخراج URL ها و Endpoint های موجود در فایل های JavaScript

- -o: ذخیره خروجی در فایل مشخص شده

بهترین شیوه اجرا

□ ابتدا خزش روی دامنه اصلی انجام شود .

□ در صورت وجود صفحات احراز هویت، با دسترسی مجاز و در محدوده تست، بخش های وارد شده نیز بررسی شوند .

□ خروجی قبل از استفاده در مراحل بعدی :

- Duplicate ها حذف شوند .

- URL های غیر مرتبط جدا شوند .

- Endpoint های حساس دسته بندی شوند.

□ URL های داخلی استخراج شدند.

□ فایل های JavaScript شناسایی شدند.

□ Endpoint های موجود در منابع سمت کاربر ثبت شدند.

□ مسیرهای دارای پارامتر مشخص شدند.

□ صفحات احراز هویت و بخش های حساس بررسی شدند.

□ URL های تکراری پاکسازی شدند.

□ خروجی برای مراحل بعدی آماده شد.

خروجی مورد انتظار

- فهرست کامل URL های کشف شده

- فهرست فایل های JavaScript

- Endpoint های قابل مشاهده

- منابع خارجی استفاده شده

- URL های دارای پارامتر

- مسیرهای مهم برای بررسی های بعدی

- فایل های JavaScript مرتبط با دامنه

مرحله 6 — شناسایی در کد سمت کاربر

JavaScript Recon

هدف

تحلیل فایل های JavaScript به منظور شناسایی Endpoint ها، مسیرهای داخلی، API ها، پارامترها، کلیدها، توکن ها و سایر اطلاعاتی که ممکن است سطح حمله اپلیکیشن را گسترش دهند.

چرا این ابزار؟

LinkFinder و SecretFinder دو ابزار مکمل هستند برای استخراج Endpoint ها و URL های موجود در کد JavaScript استفاده می شود، در حالی که SecretFinder الگوهای مربوط به Secret ها، Token ها، API Key ها و اطلاعات حساس را شناسایی می کند.

ابزار پیشنهادی

LinkFinder و SecretFinder

دستور اجرا 1

```
python3 linkfinder.py -i urls.txt -o cli
```

توضیح دستور

- -i: فایل ورودی شامل URL های فایل های JavaScript

- -o cli :نمایش خروجی در ترمینال

دستور اجرا 2

```
python3 SecretFinder.py -i urls.txt -o cli
```

توضیح دستور

- -i: فایل ورودی
- -o cli :نمایش خروجی در ترمینال
- فایل ورودی شامل لیست فایل‌های JavaScript استخراج‌شده از مرحله Crawling است.

بهترین شیوه اجرا

- فقط فایل‌های JavaScript استخراج‌شده در مرحله Web Crawling بررسی شوند .
- نتایج LinkFinder و SecretFinder به‌صورت دستی اعتبارسنجی شود زیرا ممکن است False Positive تولید شود .
- Endpoint ها و اطلاعات مهم در یک فایل مجزا مستندسازی شوند تا در مراحل API Recon و Parameter Discovery مورد استفاده قرار گیرند .

□فایل‌های JavaScript بررسی شدند.

□Endpointهای استخراج‌شده ثبت شدند.

□مسیرهای API شناسایی شدند.

□پارامترهای قابل مشاهده ثبت شدند.

□کلیدها یا Token های احتمالی بررسی شدند.

□اطلاعات حساس Hardcoded بررسی شد.

□نتایج مشکوک به‌صورت دستی اعتبارسنجی شدند.

خروجی مورد انتظار:

- فهرست فایل‌های JavaScript بررسی‌شده
- Endpointهای استخراج‌شده
- URLهای داخلی
- مسیرهای API
- پارامترهای مشاهده‌شده
- کلیدها یا Token های احتمالی
- اطلاعات حساس احتمالی (Hardcoded Secrets)
- موارد نیازمند بررسی دستی
- موارد مشکوک به False Positive

مرحله 7 — کشف محتوا

Content Discovery

هدف

شناسایی دایرکتوری‌ها، فایل‌ها، صفحات، پل‌ها و Endpoint هایی که از طریق Web Crawling قابل مشاهده نبوده‌اند و تنها با درخواست مستقیم به سرور قابل کشف هستند.

چرا این ابزار؟

ابزار پیشنهادی

ffuf یکی از سریع‌ترین و دقیق‌ترین ابزارهای Content Discovery است که امکان Fuzzing مسیرها، فیلتر کردن پاسخ‌ها و استفاده از Wordlist های مختلف را فراهم می‌کند. انعطاف بالا و سرعت مناسب، آن را به ابزار اصلی این مرحله تبدیل کرده است.

ffuf

دستور اجرا

```
ffuf -u https://target.com/FUZZ -w wordlist.txt -mc 200,204,301,302,307,401,403 -o result.json
```

توضیح دستور

- **-u**: آدرس هدف؛ عبارت FUZZ محل جایگذاری کلمات Wordlist است.
- **-w**: مسیر Wordlist مورد استفاده.
- **-mc**: فقط پاسخ‌هایی با Status Code مشخص نمایش داده شوند.
- **-o**: ذخیره نتایج در فایل.

بهترین شیوه اجرا

- ابتدا از Wordlist کوچک و عمومی استفاده شود.
- در صورت نیاز، Wordlist های تخصصی‌تر اجرا شوند.
- قبل از تحلیل نتایج، پاسخ‌های پیش‌فرض و False Positive ها حذف شوند.
- مسیرهای کشف‌شده بر اساس اهمیت و نوع پاسخ اولویت‌بندی شوند.

□ دایرکتوری‌های جدید شناسایی شدند.

□ فایل‌های قابل دسترس شناسایی شدند.

□ صفحات مدیریتی احتمالی بررسی شدند.

□ فایل‌های Backup یا Config احتمالی بررسی شدند.

□ Endpoint های جدید ثبت شدند.

□ Status Code هر مسیر ثبت شد.

□ اندازه پاسخ (Response Size) بررسی شد.

□ موارد False Positive حذف شدند.

خروجی مورد انتظار:

- Directory های کشف‌شده
- فایل‌های کشف‌شده
- صفحات مدیریتی احتمالی
- فایل‌های Backup یا Config
- Endpoint های جدید
- Status Code هر مسیر
- Response Size
- مسیرهای نیازمند بررسی دستی

Parameter Discovery

هدف	شناسایی پارامترهای ورودی برنامه، شامل پارامترهای موجود در URL، پارامترهای آرسیوشده و پارامترهای پنهانی که مستقیماً در رابط کاربری قابل مشاهده نیستند.
چرا این ابزار؟	ParamSpider و Arjun ابزارهای مکمل هستند ParamSpider با استفاده از منابع آرسیوی و داده‌های عمومی، پارامترهای شناخته‌شده را استخراج می‌کند؛ در حالی که Arjun با ارسال درخواست‌های فعال، وجود پارامترهای پنهان را روی Endpoint های مشخص بررسی می‌کند.
ابزار پیشنهادی	ParamSpider و Arjun

دستور اجرا 1

```
paramspider -d target.com
```

توضیح دستور

- **-d**: دامنه هدف برای استخراج پارامترهای آرسیوشده.

دستور اجرا 2

```
arjun -u https://target.com/endpoint -o arjun_result.json
```

توضیح دستور

- **-u**: آدرس Endpoint مورد بررسی.
- **-o**: ذخیره نتایج در فایل.

بهترین شیوه اجرا

- ابتدا با ParamSpider فهرست پارامترهای شناخته‌شده استخراج شود.
- سپس Endpoint های مهم با Arjun بررسی شوند.
- نتایج هر دو ابزار در یک فایل واحد تجمیع و موارد تکراری حذف شوند.
- پارامترهای کشف‌شده بر اساس حساسیت و کاربرد اولویت‌بندی شوند.

□ پارامترهای GET استخراج شدند.

□ پارامترهای POST احتمالی شناسایی شدند.

□ پارامترهای آرسیوشده استخراج شدند.

□ پارامترهای پنهان بررسی شدند.

□ Endpoint های دارای پارامتر ثبت شدند.

□ پارامترهای تکراری حذف شدند.

□ نتایج به‌صورت دستی اعتبارسنجی شدند.

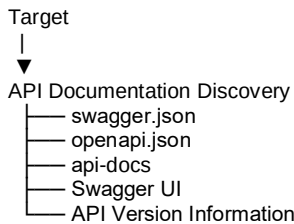
خروجی مورد انتظار

- فهرست پارامترهای GET
- فهرست پارامترهای POST احتمالی
- Endpoint های دارای پارامتر

- پارامترهای آرسو شده
- پارامترهای پنهان کشف شده
- پارامترهای نیازمند بررسی امنیتی
- موارد نیازمند بررسی دستی

مرحله 9 — شناسایی رابط برنامه نویسی

API Recon



اگر مستندات پیدا نشد:



هدف

شناسایی رابط‌های برنامه‌نویسی موجود در برنامه، کشف Endpoint های API ، بررسی مستندات احتمالی، نسخه‌های API ، متدهای HTTP و ساختار ارتباط بین Client و Server.

چرا این ابزار؟

API Recon ترکیبی از بررسی دستی و ابزارهای تخصصی است. ابتدا مستندات API مانند Swagger و OpenAPI بررسی می‌شوند، زیرا دقیق‌ترین اطلاعات درباره ساختار API را ارائه می‌دهند. در صورت نبود مستندات، Kiterunner برای کشف Endpoint های API با استفاده از Wordlist های تخصصی API استفاده می‌شود.

ابزار پیشنهادی

Kiterunner

دستور اجرا

```
kr scan https://target.com -A apiroutes-260227 -x 10 -j 1
```

توضیح دستور

- **-A**: استفاده از Wordlist مخصوص API Route ها.
- **-x 10**: تعداد درخواست‌های همزمان برای افزایش سرعت اجرا.
- **-j 1**: تعداد Host های مورد پردازش.
- در تست Narrow معمولاً فقط یک Host بررسی می‌شود.

بهترین شیوه اجرا

- ابتدا Endpoint های استخراج شده از مراحل قبلی بررسی شوند :



- مستندات API قبل از اجرای Discovery بررسی شوند .
- Endpoint های پیدا شده بر اساس اهمیت دسته‌بندی شوند :

Authentication ○

- User Management
- File Upload
- Admin
- Sensitive Functions

• پاسخ‌های دارای Status Code مهم به‌صورت دستی بررسی شوند.

□ فایل‌های Swagger/OpenAPI بررسی شدند.

□ نسخه‌های API شناسایی شدند.

□ Endpointهای API استخراج شدند.

□ متدهای HTTP ثبت شدند.

□ Endpointهای Authentication مشخص شدند.

□ Endpointهای حساس مشخص شدند.

□ APIهای مستند و غیرمستند ثبت شدند.

□ نتایج Kiterunner اعتبارسنجی دستی شدند.

خروجی مورد انتظار

- فهرست Endpointهای API
- متدهای HTTP قابل مشاهده (GET / POST / PUT / DELETE)
- نسخه‌های API
- فایل‌های مستندات API
- مسیرهای مستند و غیرمستند
- Endpointهای مرتبط با Authentication
- پارامترهای API
- موارد نیازمند بررسی امنیتی

مرحله 10 — نقشه‌برداری احراز هویت

Authentication Mapping

هدف

شناسایی و مستندسازی کامل فرآیندهای احراز هویت، مدیریت نشست کاربران، نقش‌های کاربری و نحوه تعامل Client و Server در بخش‌های حساس برنامه.

چرا این ابزار؟

این مرحله نیازمند تحلیل رفتار واقعی برنامه است و بخش زیادی از آن به بررسی دستی درخواست‌ها و پاسخ‌ها نیاز دارد. Burp Suite امکان رهگیری، اصلاح، ارسال مجدد و مقایسه درخواست‌های HTTP را در یک محیط واحد فراهم می‌کند.

ابزار پیشنهادی

Burp Suite

بهترین شیوه اجرا

- ابتدا فرآیندهای اصلی با یک کاربر عادی بررسی شوند .
- درخواست‌ها و پاسخ‌های مربوط به Login ، Logout و Password Reset ذخیره شوند .
- تفاوت رفتار سیستم در شرایط مختلف مقایسه شود :

مثال:

- Loginموفق

- Login ناموفق
- رمز اشتباه
- کاربر بدون دسترسی
- Session منقضی شده
- Token ها و Cookie های ایجاد شده ثبت و نحوه نگهداری آن ها بررسی شوند.

Login Flow

بررسی:

- Endpoint ورود شناسایی شد.
- Method درخواست (GET/POST) ثبت شد.
- پارامترهای ورودی ثبت شدند.
- Cookie یا Token ایجاد شده ثبت شد.
- رفتار Login موفق و ناموفق مقایسه شد.

Logout Flow

بررسی:

- Endpoint خروج مشخص شد.
- رفتار Session بعد از Logout بررسی شد.
- Token یا Cookie بعد از خروج بررسی شد.

Password Reset Flow

بررسی:

- مسیر بازیابی رمز مشخص شد.
- روش ارسال Token یا کد تایید ثبت شد.
- مراحل تغییر رمز مستندسازی شد.

Session Management

بررسی:

- نوع Session مشخص شد.
- محل نگهداری Session بررسی شد.
- Cookie های مرتبط ثبت شدند.
- ویژگی های Cookie بررسی شدند:

Secure

HttpOnly

SameSite

- Session Timeout بررسی شد.

Role Behavior

بررسی:

- ☐ نقش‌های کاربری قابل مشاهده ثبت شدند.
- ☐ تفاوت دسترسی نقش‌ها مشخص شد.
- ☐ صفحات حساس هر نقش ثبت شدند.

خروجی مورد انتظار

- Login کامل Flow
- Logout کامل Flow
- Flow بازیابی رمز عبور
- نوع Session Management
- Cookie ها و Token های مرتبط
- نقش‌های کاربری قابل مشاهده
- Endpoint های Authentication
- نقاط نیازمند بررسی امنیتی در فاز Vulnerability Assessment

مرحله 11 — Security Controls Mapping

موارد بررسی

سمت سرور

- ☐ WAF شناسایی شد.
- ☐ Rate Limiting بررسی شد.
- ☐ Account Lockout بررسی شد.
- ☐ IP Blocking بررسی شد.
- ☐ Bot Protection بررسی شد.

سمت احراز هویت

- ☐ MFA بررسی شد.
- ☐ CAPTCHA بررسی شد.
- ☐ Session Protection بررسی شد.

سمت مرورگر و Header ها

- ☐ CSRF Protection بررسی شد.
- ☐ CSP بررسی شد.
- ☐ HSTS بررسی شد.
- ☐ Security Headers بررسی شدند.

خروجی مورد انتظار

- مکانیزم‌های دفاعی شناسایی شده
- Headerهای امنیتی فعال
- محدودیت‌های درخواست
- کنترل‌های مرتبط با Authentication

مرحله 12 — جمع‌بندی یافته‌های Recon Recon Summary & Documentation

Final Checklist

پیش از ورود به مرحله ارزیابی آسیب‌پذیری باید موارد زیر آماده باشند:

- ☐ مجوز و Scope تست ثبت شده است.
- ☐ معماری کلی برنامه و مسیرهای اصلی مشخص شده‌اند.
- ☐ صفحات و قابلیت‌های اصلی برنامه مستند شده‌اند.
- ☐ گزارش HTTP Headers و رفتار پاسخ سرور آماده است.
- ☐ وضعیت SSL/TLS و Certificate ثبت شده است.
- ☐ فناوری‌های استفاده‌شده شناسایی شده‌اند.
- ☐ URLها و فایل‌های JavaScript استخراج شده‌اند.
- ☐ Endpointهای استخراج‌شده از JavaScript ثبت شده‌اند.
- ☐ Directoryها و فایل‌های کشف‌شده ثبت شده‌اند.
- ☐ پارامترهای شناسایی‌شده جمع‌آوری شده‌اند.
- ☐ Endpointهای API و مستندات احتمالی ثبت شده‌اند.
- ☐ Flow احراز هویت و Session Management مستند شده است.
- ☐ کنترل‌های امنیتی شناسایی شده‌اند.